I-Logical Approaches Logical Inference and Message Passing Algorithms

Foundations of Neuro-Symbolic AI

Alex Goessmann

University of Applied Science Würzburg-Schweinfurt (THWS)

April 13, 2026

Outline

- Monotonicity of Entailment
- Syntactical Inference Rules

Motivation

Contractions of large knowledge bases get infeasible.

Is it really necessary to perform large contractions?

Example: Entailment in the toy accounting system

When knowing that feature X_F is true, we can conclude that X_{A1} is true, without considering other accounting rules.

Outlook: Inference algorithms

Inference algorithms focus on subsets of knowledge bases.
We now show that:

- **Soundness:** "Every by a sub knowledge base entailed formula is entailed by the knowledge base itself."
- **Memory:** "Every entailed formula can be added to the knowledge base."

Monotonicity of Entailment

Theorem (Monotonicity of Entailment)

Given a knowledge base $\mathcal{KB}$ of formulas, let $\tilde{\mathcal{KB}} \subset \mathcal{KB}$ be a subset and let f be another propositional formula. If

$$\tilde{\mathcal{KB}} \models f$$

then also

$$\mathcal{KB} \models f$$

Soundness but no Completeness

Notice that the back direction does not hold: When $\mathcal{KB} \models f$, then $\tilde{\mathcal{KB}} \models f$ does not always hold.

Invariance of adding entailled formulas

When $\mathcal{KB} \models f$, then

$$\mathcal{KB}\left[X_{[d]}\right] = \left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right] \right\rangle_{\left[X_{[d]}\right]}$$

Interpretation

We can add memory formulas to a knowledge base.

Support tensors

Support tensor to a tensor $\tau\left[X_{[d]}\right]$: $\mathbb{I}_{\neq 0}\left(\tau\left[X_{[d]}\right]\right)$ with coordinates

$$\mathbb{I}_{\neq 0}\left(\tau\left[X_{[d]} = x_{[d]}\right]\right) = \begin{cases} 1 & \text{if} \quad \tau\left[X_0 = x_0, \ldots, X_{d-1} = x_{d-1}\right] \neq 0 \\ 0 & \text{else} \end{cases}$$

Invariance under adding support tensor to sub-contractions

Any support tensor of a sub-contraction can be added to a tensor network: When $\tilde{\mathcal{G}}$ a sub-hypergraph of $\mathcal{G}$, then

$$\left\langle \tau^{\mathcal{G}}\left[X_{[d]}\right]\right\rangle_{[X_{\mathcal{V}}]} = \left\langle \tau^{\mathcal{G}}\left[X_{[d]}\right], \mathbb{I}_{\neq 0}\left(\left\langle \tau^{\tilde{\mathcal{G}}}\left[X_{\mathcal{V}}\right]\right\rangle_{[X_{\mathcal{U}}]}\right)\right\rangle_{[X_{\mathcal{V}}]}$$

In propositional logics, this means:

$$\mathbb{I}_{\neq 0}\left(\left\langle \tau^{\mathcal{G}}\left[X_{[d]}\right]\right\rangle_{[X_{\mathcal{V}}]}\right) \models \mathbb{I}_{\neq 0}\left(\left\langle \tau^{\tilde{\mathcal{G}}}\left[X_{\mathcal{V}}\right]\right\rangle_{[X_{\mathcal{U}}]}\right)$$

Inference algorithms based on sub-contractions

Computation by sub-contractions (including previous messages)

Problem: Many possible sub-contractions

In large tensor networks, there are many possible sub-contractions. For efficiency, different scheduling policies emerge:

- Syntactical inference algorithms (today)
- Message passing algorithms (next lecture)

Outline

- Monotonicity of Entailment
- Syntactical Inference Rules

Inference using Syntax

So far: Inference and Entailment is formulated by semantics (with tensor interpretations).
Syntactical Inference Rules find entailed formulas using syntactical representations only.
Notation: When f is entailed by $\tilde{\mathcal{KB}}$ we denote

$$\frac{\overbrace{\tilde{\mathcal{KB}}}^{\text{has to be known for the rule to apply}}}{\underbrace{f}_{\text{is added to the knowledge base}}}$$

Clauses

Recall: Given $\mathcal{U} \subset [d]$ and $y_{\mathcal{U}} \in \times_{k \in \mathcal{U}} [2]$ the corresponding clause is

$$Z^{\vee}_{(\mathcal{U}, y_{\mathcal{U}})} [X_{\mathcal{U}}] = \mathbb{I} [X_{\mathcal{U}}] - \epsilon_{y_{\mathcal{U}}} [X_{\mathcal{U}}]$$

Special types:

- Horn clause: There is at most one $k \in \mathcal{U}$ with $y_k = 1$
- Definite clause: There is at exactly one $k \in \mathcal{U}$ with $y_k = 1$

Interpretation of definite clauses: Implications Example:

$$\begin{aligned} \left(X_F \wedge X_{GP1} \rightarrow X_{MAT} \right) &\Leftrightarrow \neg \left((X_F \wedge X_{GP1}) \vee X_{MAT} \right) \\ &\Leftrightarrow \left(\neg X_F \vee \neg X_{GP1} \vee X_{MAT} \right) \end{aligned}$$

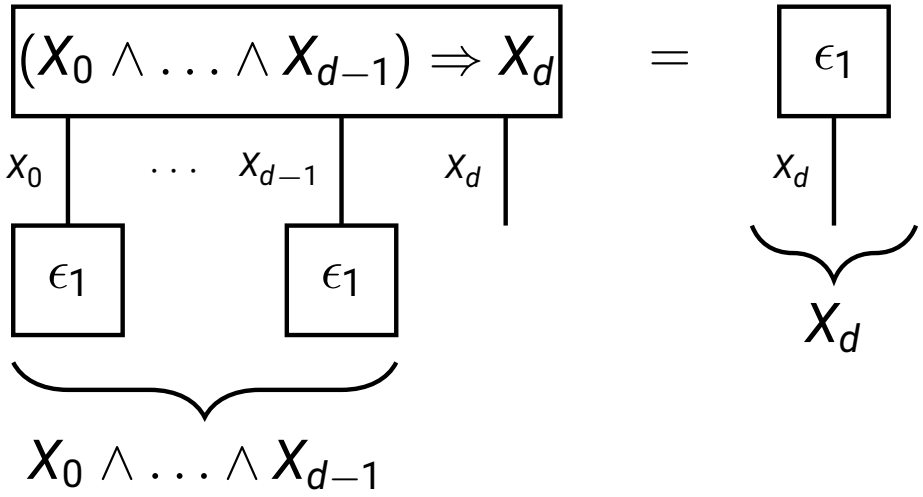
Modus Ponens

Modus Ponens Inference Rule

Modus Ponens is the simplest inference rule: When an implication holds and the premise is true, then the head follows.

$$\frac{(X_0 \wedge \ldots \wedge X_{d-1}) \rightarrow X_d\,,\ (X_0 \wedge \ldots \wedge X_{d-1})}{X_d}$$

Modus ponens corresponds with the contraction equation:



Forward Chaining

Iterate modus ponens until the premises of none of the remaining implications are known:

Forward Chaining Algorithm

Require: Set of clauses $\mathcal{C}$

Ensure: Entailed literals $\mathcal{L}$, remaining set of clauses

Initialize $\mathcal{L} = \varnothing$

while exists a clause $(\{v, \tilde{y}_v\})$ of length 1 in $\mathcal{C}$ **do**

  for all clauses $(\mathcal{U}, y_{\mathcal{U}})$ with $v \in \mathcal{U}$ **do**

    if $y_v \neq \tilde{y}_v$ **then**

      $(\mathcal{U}, y_{\mathcal{U}}) \leftarrow (\mathcal{U} \backslash \{v\}, y_{\mathcal{U} \backslash \{v\}})$

    else

      $\mathcal{C}.\mathrm{drop}((\mathcal{U}, y_{\mathcal{U}}))$

    end if

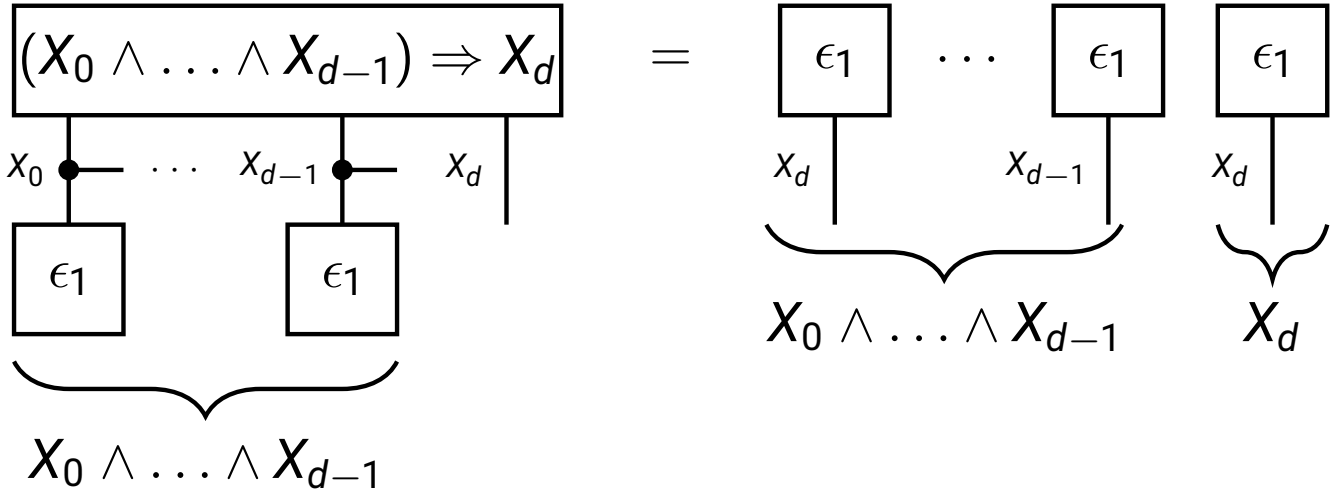
  end for

  $\mathcal{L}.\mathrm{append}(\{v, \tilde{y}_v\})$

end while

Modus ponens as removing edges

We now keep also the colors to the premises open:



Thus we understand modus ponens as a method to turn tensors to elementary tensors.

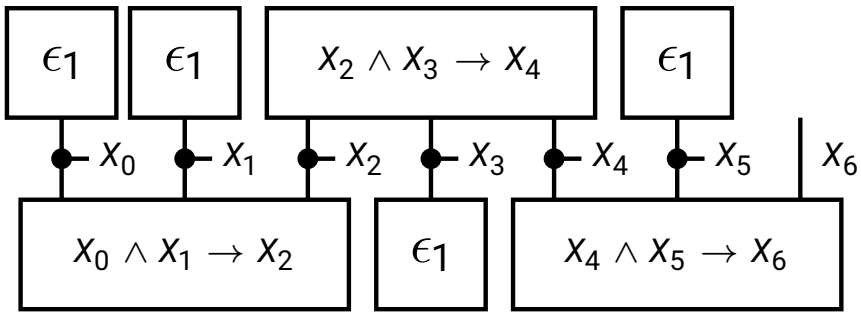
Forward chaining as disentangling the network

"Disentangling": Hyperedges connecting many edges are replaced by single node edges

Single node edges are then iteratively contracted

Example:

-
-
-



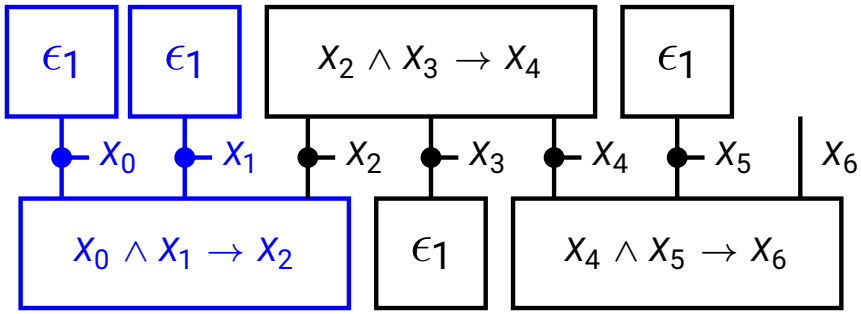
Forward chaining as disentangling the network

"Disentangling": Hyperedges connecting many edges are replaced by single node edges

Single node edges are then iteratively contracted

Example:

- Inference using $X_0 \wedge X_1 \Rightarrow X_2$
-
-



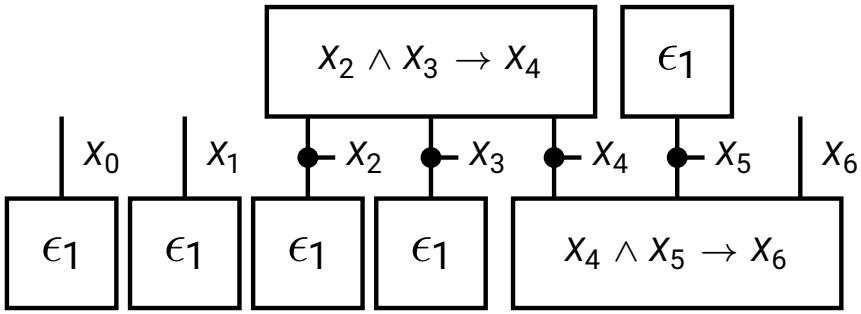
Forward chaining as disentangling the network

"Disentangling": Hyperedges connecting many edges are replaced by single node edges

Single node edges are then iteratively contracted

Example:

- Inference using $X_0 \wedge X_1 \Rightarrow X_2$
-
-



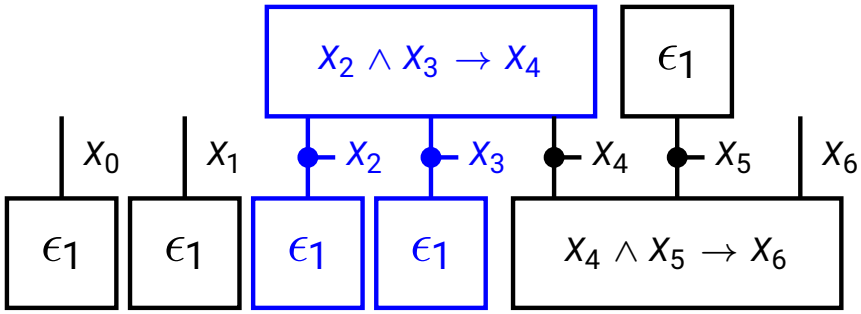
Forward chaining as disentangling the network

"Disentangling": Hyperedges connecting many edges are replaced by single node edges

Single node edges are then iteratively contracted

Example:

- Inference using $X_0 \wedge X_1 \Rightarrow X_2$
- Inference using $X_2 \wedge X_3 \Rightarrow X_4$
-



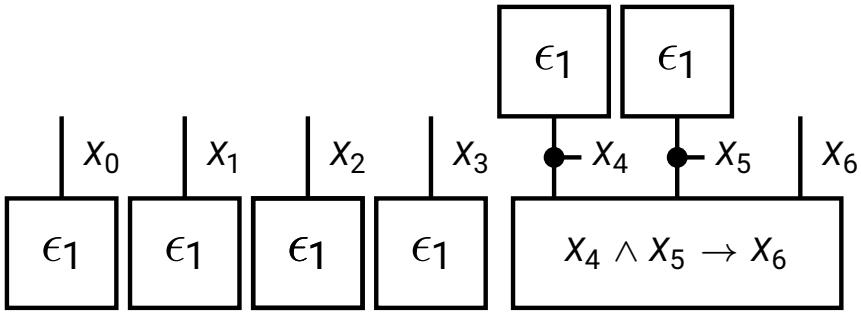
Forward chaining as disentangling the network

"Disentangling": Hyperedges connecting many edges are replaced by single node edges

Single node edges are then iteratively contracted

Example:

- Inference using $X_0 \wedge X_1 \Rightarrow X_2$
- Inference using $X_2 \wedge X_3 \Rightarrow X_4$
-



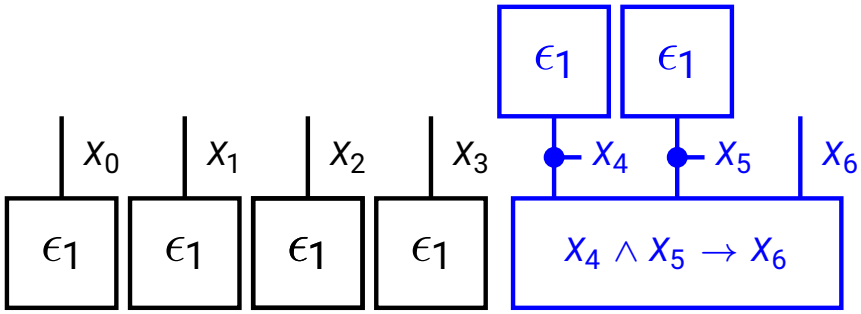
Forward chaining as disentangling the network

"Disentangling": Hyperedges connecting many edges are replaced by single node edges

Single node edges are then iteratively contracted

Example:

- Inference using $X_0 \wedge X_1 \Rightarrow X_2$
- Inference using $X_2 \wedge X_3 \Rightarrow X_4$
- Inference using $X_4 \wedge X_5 \Rightarrow X_6$



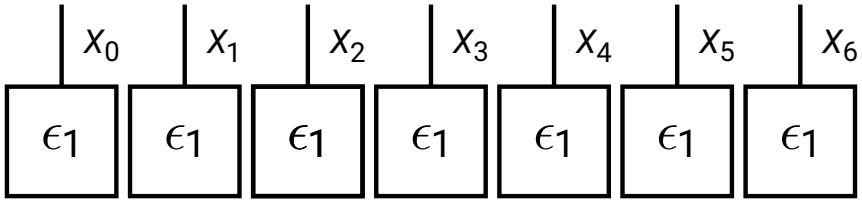
Forward chaining as disentangling the network

"Disentangling": Hyperedges connecting many edges are replaced by single node edges

Single node edges are then iteratively contracted

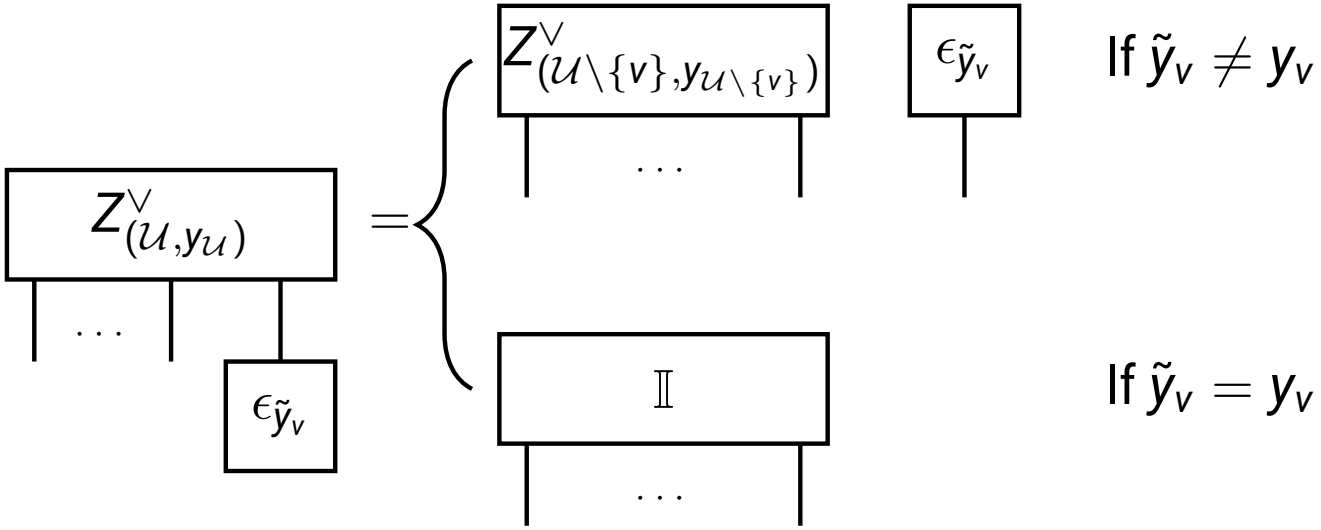
Example:

- Inference using $X_0 \wedge X_1 \Rightarrow X_2$
- Inference using $X_2 \wedge X_3 \Rightarrow X_4$
- Inference using $X_4 \wedge X_5 \Rightarrow X_6$



Contraction of clauses

Underlying is the contraction



Therefore:

- When clauses "disagreeing" on v: Larger clause shrinks
- When clauses "agreeing" on v: Larger clause becomes trivial

More general: Contraction of clauses

When contracting a clause (where $k \in \mathcal{U}$) with a basis vector $\epsilon_{x_k}[X_k]$ we have two possible situations:

- The clause becomes trivial, if $x_k \neq y_k$
- The clause shrinks, if $x_k = y_k$

Definite Clauses

Lemma

A knowledge base of definite clauses of length at least 2 always has the states $1_{\mathcal{V}}$ and $0_{\mathcal{V}}$ as models.

Proof sketch: By using the contraction of clauses properties we know that:

- When contracting with $\bigtimes_{v \in \mathcal{V}} \epsilon_1 [X_v]$, all clauses become trivial.
- When contracting with $\bigtimes_{v \in \mathcal{V}} \epsilon_0 [X_v]$, all clauses become trivial.

Thus, $x_{\mathcal{V}} = 1_{\mathcal{V}}$ and $x_{\mathcal{V}} = 0_{\mathcal{V}}$ are models of the knowledge base.

Forward chaining is complete on definite clauses

Theorem

When $\mathcal{KB}$ is all clauses in are definite, i.e. have exactly one positive literal, then $\mathcal{L}$ contains all entailed literals

Sketch of the proof:

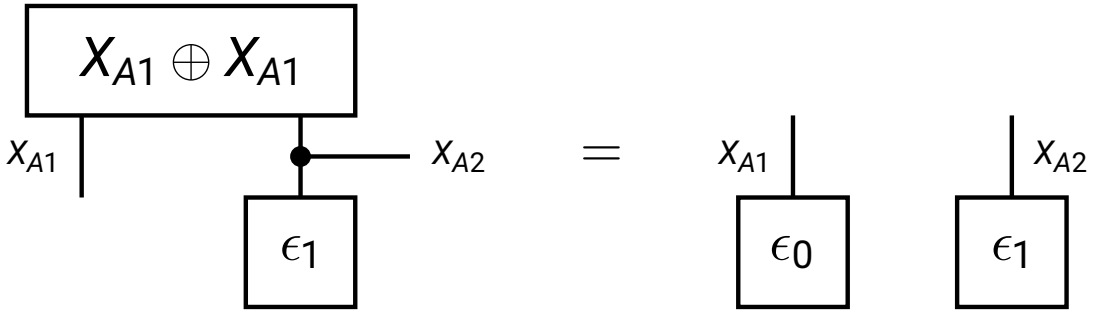
- Forward chaining on definite clauses terminates with a knowledge base of definite clauses of length at least 2
- The final knowledge base has $1_{\mathcal{V}}$ and $0_{\mathcal{V}}$ as a model
- The final knowledge base does therefore not entail for any $v \in \mathcal{V}$ the literals X_v or $\neg X_v$

Generalization: Contraction perspective

First toy accounting rule $X_{A1} \oplus X_{A2}$ does not have a CNF of definite clauses!

$$X_{A1} \oplus X_{A2} \equiv \left(\neg X_{A1} \vee \neg X_{A2}\right) \wedge \left(X_{A1} \vee X_{A2}\right)$$

But: We can still do contractions to simplify.
Example: When knowing that X_{A2} is true, we get with this accounting that X_{A1} is false:



Generalization: Resolution

Resolution Inference Rule

$$\frac{f_0 \vee X\,,\;\; f_1 \vee \neg X}{f_0 \vee f_1}$$

Scheduling is key

Number of large contractions

The number of possible sub-contractions grows fast with the number k of the subcontraction: $\binom{|\mathcal{E}|}{k}$

Resolution:

- Restricts the possible sub-contractions to those of clauses with complementary clauses
- Requires further scheduling policies for efficiency

Selection of contractions to be performed is central to algorithm design

- Forward chaining
- SLD resolution (e.g. in ProLog)